Reinforcement Learning for Autonomous Penetration Testing: A Comparative Study of DQN and PPO with Curriculum Learning

Vinay Appajigowda
Student ID: x24262013
National College of Ireland
Dublin, Ireland
x24262013@student.ncirl.ie

***Abstract*—Automated penetration testing is essential for proactive cyber-security, yet manual assessments are time-consuming, expensive, and difficult to reproduce consistently. This paper investigates two deep reinforcement learning (RL) algorithms—Deep Q-Network (DQN) and Proximal Policy Optimization (PPO)—for autonomous network penetration testing in a custom simulated environment modelled as a Markov Decision Process (MDP). We formally specify the state space, action space, transition dynamics, and reward function, and provide complete DQN temporal-difference and PPO clipped-surrogate loss derivations. We introduce a *curriculum learning* strategy where agents are first trained on small networks (10 hosts) before weight transfer to progressively larger topologies (20, 30, and 50 hosts). Experimental results across 20 evaluation episodes per configuration demonstrate that (i) PPO achieves a mean reward of 14.27 ± 1.56 on 20-host networks versus DQN's 13.77 ± 2.21 (Cohen's $d = 0.26$, Welch's $t(36) = 0.91$, $p = 0.37$, bootstrap 95% CI for difference: $[-0.48, 1.48]$), (ii) curriculum-trained agents converge approximately 30% faster than from-scratch baselines, and (iii) both algorithms exhibit graceful performance degradation as network complexity increases, with 100% compromise ratios on up to 20 hosts. A formal scalability analysis shows that the state space grows as 3^N and action space as $O(N \times V)$, providing theoretical bounds on tractable network sizes. We discuss reward shaping sensitivity, threats to validity, transfer learning theory, and responsible-use obligations under the CFAA, the Computer Misuse Act 1990, and the NIS2 Directive.**

***Index Terms*—reinforcement learning, penetration testing, deep Q-network, proximal policy optimization, curriculum learning, Markov decision process, transfer learning, cyber-security**

I. Introduction

Penetration testing is a controlled process of exploiting vulnerabilities in computer networks to assess their security posture [1]. Traditional penetration testing relies heavily on human expertise and is both expensive and difficult to scale [2]. The global penetration-testing market is projected to exceed $4.5 billion by 2025, driven by regulatory mandates (NIS2, PCI-DSS, ISO 27001) and growing attack surfaces introduced by cloud migration and IoT proliferation. Reinforcement learning (RL) offers a promising path toward automating this process by allowing agents to learn attack strategies through interaction with simulated environments [3], [4].

Recent work has demonstrated the viability of RL for automated penetration testing. Ghanem and Chen [5] first proposed a structured RL formulation for pentesting, and subsequent hierarchical extensions [1] demonstrated efficiency gains on large networks. Zhou *et al.* [6] applied double-DQN to autonomous network penetration testing with prioritized replay, achieving strong results on networks of up to 50 nodes. More recently, Terranova *et al.* [7] introduced a rigorous generalization protocol that tests trained agents on topologies not seen during training.

Despite these advances, two challenges remain under-explored:

1) **Algorithm comparison:** Systematic side-by-side evaluation of value-based (DQN) and policy-gradient (PPO) methods under identical, formally specified conditions.
2) **Scalability via curriculum learning:** Training on small networks and transferring knowledge to larger, unseen topologies, with a formal analysis of state-space growth.

This paper addresses both gaps. Our contributions are:

- A formally specified MDP for network penetration testing, with explicit state space, action space, transition dynamics, and reward function.
- A rigorous comparison of DQN and PPO with full statistical analysis (Welch's t-test, Mann-Whitney U, Cohen's d, bootstrap 95% CIs) across multiple network sizes.
- A curriculum learning pipeline improving convergence speed by $\sim$30%, with a theoretical account based on potential-based reward shaping.
- A formal scalability analysis showing $|\mathcal{S}| = 3^N$ growth and its implications for tractable network sizes.
- A comprehensive ethics and responsible-use discussion grounded in applicable law and dual-use frameworks.

II. Related Work

A. Comparison of RL-Based Penetration Testing Approaches

Table I summarises representative RL-based penetration testing papers, providing a structured basis for positioning the present work. Several observations emerge. First, value-based methods (DQN and its variants) dominate early work, while policy-gradient methods (PPO, A3C) appear in more recent contributions. Second, evaluated network sizes remain small ($\leq$ 50 hosts in most cases), and few studies test generalization to unseen topologies. Third, reported success rates vary widely (55–100%), partly due to differing difficulty calibrations. Our work is distinctive in providing both a formal MDP specification and a systematic statistical comparison of two algorithm families.

B. Foundational Deep RL Algorithms

Value-based methods. Mnih *et al.* [11] introduced the Deep Q-Network (DQN), demonstrating human-level performance across Atari games by combining Q-learning with experience replay and target networks. The key insight—that a convolutional network can serve as a universal function approximator for Q-values—directly enabled value-based RL in high-dimensional state spaces. Van Hasselt *et al.* [12] introduced Double DQN (DDQN) to reduce overestimation bias by decoupling action selection from evaluation. Prioritized experience replay [13] further improved data efficiency by sampling transitions proportional to temporal-difference error. However, DQN's reliance on an ϵ-greedy exploration strategy is inefficient in environments with large, structured action spaces where most actions are invalid—a challenge we address through action masking.

Policy-gradient methods. Schulman *et al.* [14] proposed Proximal Policy Optimization (PPO), achieving strong empirical performance through a clipped surrogate objective that avoids expensive second-order optimization. Earlier, Mnih *et al.* [15] introduced the Asynchronous Advantage Actor-Critic (A3C), demonstrating that asynchronous parallel workers could stabilize policy-gradient training. Lillicrap *et al.* [16] extended actor-critic methods to continuous action spaces via Deep Deterministic Policy Gradient (DDPG). Haarnoja *et al.* [17] further advanced this direction with Soft Actor-Critic (SAC), which maximizes a trade-off between reward and entropy, yielding superior sample efficiency in continuous-control benchmarks. PPO's on-policy nature makes it less sample-efficient than off-policy methods like DQN; we empirically evaluate this trade-off in our penetration testing domain.

Landmark RL achievements. Silver *et al.* [18] demonstrated that deep RL combined with Monte Carlo Tree Search can surpass human expertise in Go, a game with a state space orders of magnitude larger than network pentesting environments. This result provides theoretical grounding for the hypothesis that deep RL can eventually master complex sequential decision problems such as penetration testing. The foundational treatment of RL theory is provided by Sutton and Barto [4], whose formalization of MDPs, temporal difference learning, and policy optimization underpins all algorithms evaluated in this paper.

C. RL for Penetration Testing: Critical Analysis

Ghanem and Chen [5] were among the first to formalize penetration testing as an MDP and apply tabular Q-learning and DQN. Their work demonstrates that RL agents can discover non-trivial multi-hop exploit chains, but is limited to small networks ($\leq$ 25 hosts) with deterministic exploitation—a simplification that inflates reported success rates. Their 2023 follow-up [1] introduces hierarchical RL to scale to 50-host networks; however, the hierarchy is hand-crafted by domain experts rather than learned, which reduces generalizability.

Zhou *et al.* [6] applied Double DQN with Prioritized Experience Replay to autonomous penetration testing, testing on networks with up to 50 hosts and reporting 68–85% success rates depending on vulnerability density. A notable strength is their ablation study isolating the contribution of prioritized replay (+12% success rate over uniform replay). A limitation is the use of a fixed topology across all evaluations, which prevents assessment of generalization to unseen graph structures.

Schwartz and Kurniawati [8] applied A3C to the CyberBattleSim platform, achieving 55% success on 10-host networks. Their key finding is that on-policy methods converge more stably than value-based methods in highly stochastic exploit environments. However, their evaluation is confined to very small networks and does not investigate curriculum-based transfer.

Terranova *et al.* [7] introduced a rigorous generalization protocol, reporting curriculum-based training accelerating convergence by approximately 30% on medium networks. Their finding directly motivates our curriculum learning design. A weakness is the reliance exclusively on PPO without comparison to value-based baselines.

Nguyen *et al.* [3] present PenGym, grounded in real CVE data, achieving 91% success with PPO on networks up to 20 hosts. PenGym's realism substantially raises the bar for simulation fidelity, but the closed environment prevents community comparison.

D. Foundational Methods: Curriculum and Transfer Learning

Bengio *et al.* [19] formalized curriculum learning—presenting examples from easy to hard—showing improvements in convergence speed and generalization. Their theoretical motivation, rooted in continuation methods from numerical optimization, provides a principled justification for our progressive network-size training schedule. Taylor and Stone [20] surveyed transfer learning in RL, identifying inter-task mappings, reward shaping, and policy reuse as primary transfer mechanisms. Our curriculum learning approach is most closely related to their "sequential transfer" paradigm. A key insight is that negative transfer occurs when source and target tasks differ substantially in dynamics, which motivates our design choice of maintaining consistent vulnerability distributions across curriculum phases.

TABLE I
COMPARISON OF REPRESENTATIVE RL-BASED PENETRATION TESTING WORKS

Author / Year	RL Algo.	Environment	Net. Size	Success Rate	Key Limitation
Ghanem & Chen [5] (2020)	DQN, HRL	Custom graph	5–25 hosts	72%	Small networks; no curriculum; deterministic exploits
Zhou et al. [6] (2021)	DDQN+PER	Custom graph	5–50 hosts	68%–85%	No policy-gradient comparison; static topology
Schwartz & Kurniawati [8] (2019)	A3C	CyberBattleSim	10 hosts	55%	Small scale; no transfer; on-policy only
Ghanem et al. [1] (2023)	HRL (DQN)	Custom	15–50 hosts	80%	Hierarchy hand-crafted; no statistical testing
Liu et al. [9] (2024)	HRL	Custom	15 hosts	88%	Not tested beyond 15 hosts; no ablation
Terranova et al. [7] (2024)	PPO	Custom graph	10–30 hosts	76%	No DQN comparison; synthetic topologies only
Oh et al. [10] (2024)	DQN	Custom sim.	10–20 hosts	82%	Single algo; no algorithm family comparison
Nguyen et al. [3] (2025)	PPO	PenGym (real CVEs)	5–20 hosts	91%	Linux-only; no curriculum; closed environment
Alhamed & Rahman [2] (2025)	DQN, QR-DQN	Custom DoS env	1 target	95%	Single attack type; no network topology
Ours (2025)	**DQN, PPO**	**Custom MDP**	**10–50 hosts**	**100% (10–20)**	Synthetic env.; no adaptive defence; 100% ceiling limits discrimination

Moreno *et al.* [21] combine RL with recommender systems for action-space reduction in large networks. The hybrid approach is innovative but introduces an additional supervised learning dependency on historical attack data. Sun *et al.* [22] integrate ontology knowledge with RL, reducing exploration time in specialized IoT environments. The ontology must, however, be manually curated for each target domain, reducing portability.

E. Security Frameworks and Game Theory

Kennedy *et al.* [23] describe the Metasploit framework, the most widely adopted penetration testing platform. Our simulated environment draws conceptual parallels to Metasploit's module-based exploitation model. The PTES [24] defines structured methodology encompassing pre-engagement, intelligence gathering, exploitation, and post-exploitation—phases that map directly to our scan and exploit action types.

Tambe [25] formulates network security as a Stackelberg game between attackers and defenders. While our work treats the defender as static, game-theoretic formulations offer a natural extension for modeling adaptive defences. The connection between RL and game theory is particularly relevant: a pentesting agent implicitly optimizes against a fixed-policy defender, a degenerate case of the general adversarial game. Zhu *et al.* [26] provide a comprehensive survey of RL for autonomous cyber defence, identifying scalability and sim-to-real transfer as the two most critical open challenges—both of which we address directly.

III. METHODOLOGY

A. Formal MDP Specification

We model the penetration testing task as a discrete-time, finite-horizon Markov Decision Process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma, T)$ with the following components.

State space. Each host h_i is in one of three states: unknown (0), scanned (1), or compromised (2). Additionally, each vulnerability v_j is either known (1) or unknown (0). The global state is:

$$\mathcal{S} = \big\{ s \mid s = (h_1, \ldots, h_N, k_1, \ldots, k_V, e_1, \ldots, e_N) \big\} \quad (1)$$

where $h_i \in \{0, 1, 2\}$ denotes host i's compromise level, $k_j \in \{0, 1\}$ denotes whether vulnerability j has been discovered, and $e_i \in \{0, 1\}$ is an auxiliary entry-point flag. Considering only the host-compromise component, $|\mathcal{S}| = 3^N$, which grows exponentially with network size (see Section V-E).

Action space. The agent has access to two action types:

$$\mathcal{A} = \big\{ \texttt{scan}(h_i) \mid i \in [N] \big\} \cup \big\{ \texttt{exploit}(h_i, v_j) \mid i \in [N],\, j \in [V_i] \big\} \quad (2)$$

where V_i is the set of vulnerabilities assigned to host h_i. The total action-space size is $|\mathcal{A}| = N + \sum_{i=1}^{N} V_i = O(N \times \bar{V})$, where $\bar{V}$ is the mean vulnerabilities per host. Invalid actions (scanning a compromised host, exploiting an unknown vulnerability) are masked to $-\infty$ in DQN and to zero probability in PPO.

Transition function. The environment dynamics are stochastic:

$$P(s' \mid s, \texttt{exploit}(h_i, v_j)) = \begin{cases} 1 - d_j & \text{host } h_i \text{ newly compromised} \\ d_j & \text{state unchanged (exploit failed)} \end{cases} \quad (3)$$

where $d_j \in [0.2, 0.95]$ is the difficulty parameter of vulnerability j. Scan actions are deterministic: $P(s' \mid s, \texttt{scan}(h_i)) = 1$ with h_i transitioning to state 1 and all associated vulnerabilities revealed.

Reward function. The reward signal balances objective achievement against operational cost:

$$R(s, a, s') = r_{\text{comp}}(s, s') - c_{\text{step}} - c_{\text{detect}}(a) \quad (4)$$

where $r_{\text{comp}}(s, s') = +1.0$ if s' contains a newly compromised host and $+5.0$ bonus on full-network compromise; $c_{\text{step}} = 0.1$ is a per-step time penalty; $c_{\text{detect}}(a) = 0$ in the base environment (detection modelling is discussed as future work in Section IX).

Discount factor and termination. We use $\gamma = 0.99$, balancing long-horizon planning against near-term reward. An episode terminates when all hosts are compromised (success) or when the step limit $T = 10 \times N$ is exceeded (failure).

B. DQN Agent

Our DQN implementation follows Mnih *et al.* [11]. The Q-network $Q_\theta(s, a)$ is trained to minimize the temporal-difference loss:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\big)^2\Big] \tag{5}$$

where $\mathcal{D}$ is the experience replay buffer and θ^- are the parameters of a periodically synchronized target network. The expectation is approximated by mini-batch sampling.

Action masking. Invalid actions receive $Q_\theta(s, a) = -\infty$ prior to the $\arg\max$ in both action selection and target computation, preventing invalid actions from influencing gradient updates.

Architecture. Two-layer MLP with 128 hidden units each, ReLU activations, and a linear output layer of size $|\mathcal{A}|$.

Exploration. ϵ-greedy with exponential decay from $\epsilon_0 = 1.0$ to $\epsilon_{\min} = 0.01$ over 5,000 training steps:

$$\epsilon_t = \epsilon_{\min} + (\epsilon_0 - \epsilon_{\min}) \cdot e^{-t/\tau}, \quad \tau = 1000 \tag{6}$$

C. PPO Agent

Our PPO implementation follows Schulman *et al.* [14]. Define the probability ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$. The clipped surrogate objective is:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big] \tag{7}$$

where $\epsilon = 0.2$ is the clipping threshold and $\hat{A}_t$ is the generalized advantage estimate (GAE) [27]:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \tag{8}$$

with $\lambda = 0.95$ controlling the bias-variance trade-off. The full PPO objective combines policy, value, and entropy terms:

$$L(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_v \mathcal{L}_V(\phi) + \beta \mathcal{H}[\pi_\theta(\cdot|s_t)] \tag{9}$$

where $\mathcal{L}_V(\phi) = \hat{\mathbb{E}}_t[(V_\phi(s_t) - V_t^{\text{targ}})^2]$, $c_v = 0.5$ is the value loss coefficient, and $\beta = 0.01$ is the entropy coefficient.

Architecture. Separate actor and critic MLPs, each with two hidden layers of 128 units and ReLU activations. The actor outputs a masked softmax distribution over valid actions; the critic outputs a scalar state value $V_\phi(s)$.

D. Curriculum Learning

Motivated by curriculum learning theory [19] and its application in RL pentesting [3], [7], we train agents in two phases:

1) **Phase 1 (small):** 300 episodes on a 10-host network.
2) **Phase 2 (medium):** 300 episodes on a 20-host network, with weights transferred from Phase 1.

The agent is then evaluated on unseen 30- and 50-host networks. Weight transfer uses partial copying: shared dimensions are copied directly; new input/output dimensions are randomly initialized with Xavier uniform initialization [28].

Curriculum design rationale. From the perspective of potential-based reward shaping [29], the smaller network implicitly defines a potential function $\Phi(s)$ proportional to the fraction of hosts already compromised. The shaping reward $F(s, s') = \gamma\Phi(s') - \Phi(s)$ accelerates learning by providing dense guidance toward the goal state. When the agent transfers to a larger network, the learned Φ provides a warm initialization that biases exploration toward productive regions of the enlarged state space—a theoretical account consistent with Taylor and Stone's sequential transfer paradigm [20].

IV. Experimental Setup

All experiments use PyTorch 2.1.0 on a single CPU (Intel Core i7-12700H, 16 GB RAM, Ubuntu 22.04). Table II provides the complete hyperparameter specification for both algorithms.

Network sizes tested: 10, 20, 30, and 50 hosts. Each evaluation consists of 20 episodes with greedy (DQN) or mode-action (PPO) selection. Curriculum training uses Phase 1 (10-host, 300 episodes) followed by Phase 2 (20-host, 300 episodes) with weight transfer.

V. Results and Discussion

A. Training Performance and Learning Curve Analysis

Fig. 1 shows training curves for DQN and PPO on a 10-host network. Both algorithms learn to compromise the majority of hosts within 200 episodes. PPO exhibits lower per-episode reward variance throughout training, reflecting the on-policy clipped objective's stabilizing effect. DQN's convergence is characterized by an initial plateau (episodes 1–80) corresponding to the ϵ-greedy exploration phase, followed by rapid improvement as ϵ decays and the replay buffer populates.

Convergence speed. We define convergence as the first episode at which a 20-episode rolling mean success rate exceeds 90%. On the 10-host network, DQN converges at episode $\approx$110 and PPO at episode $\approx$95—a 14% advantage for PPO in sample efficiency. On the 20-host network, DQN converges at episode $\approx$185 versus PPO's $\approx$160 ($\approx$14% faster). The curriculum-trained variants converge at episodes $\approx$130 (DQN) and $\approx$115 (PPO) on the 20-host task, representing $\approx$30% and $\approx$28% speedups respectively, consistent with the theoretical predictions of Bengio *et al.* [19] and the empirical findings of Terranova *et al.* [7].

TABLE II
COMPLETE HYPERPARAMETER SETTINGS FOR DQN AND PPO

Parameter	DQN	PPO
Network Architecture		
Hidden layers	128–128	128–128 (actor + critic)
Activation	ReLU	ReLU
Output	Linear ($\|\mathcal{A}\|$)	Softmax / Linear
Optimization		
Optimizer	Adam	Adam
Learning rate	1×10^{-3}	3×10^{-4} (actor) 1×10^{-3} (critic)
Gradient clipping	1.0	0.5
RL-Specific		
Discount γ	0.99	0.99
Batch size	64	64
Training episodes	400	400
DQN-Specific		
Replay buffer size	50,000	—
Min. buffer fill	1,000	—
Target net. update	every 500 steps	—
ϵ_0 / $\epsilon_{\min}$	1.0 / 0.01	—
ϵ decay steps	5,000	—
Decay constant τ	1,000	—
PPO-Specific		
Clip threshold ϵ	—	0.2
PPO epochs per update	—	4
Rollout length	—	2,048 steps
GAE λ	—	0.95
Value coeff. c_v	—	0.5
Entropy coeff. β	—	0.01
Environment		
Vuln. difficulty	$d \sim \mathcal{U}[0.2, 0.95]$	
Vulns. per host	$\sim \mathcal{U}\{1, 2, 3\}$	
Graph model	Erdős–Rényi	
Connection prob.	$p = 0.3$ (10h), 0.2 (20h), 0.15 (30–50h)	
Random seeds	42, 123, 456	

Sample efficiency comparison. PPO's on-policy nature means each sample is used only once per update, whereas DQN's experience replay buffer allows each transition to be replayed multiple times. Despite this apparent efficiency advantage for DQN, PPO converges faster in terms of episodes, suggesting that the clipped objective's variance reduction more than compensates for the lack of replay. This finding aligns with the observations of Schulman *et al.* [14] in game environments with structured action spaces.

B. Comprehensive Statistical Analysis

Table III presents evaluation results computed from $n = 20$ episodes per configuration. We employ four statistical measures to characterize the DQN vs. PPO comparison rigorously.

Per-episode rewards from eval_results.json.

- **10-host DQN:** $\bar{x} = 11.15$, $s = 0.49$; lengths $\bar{l} = 28.5$
- **10-host PPO:** $\bar{x} = 11.28$, $s = 0.41$; lengths $\bar{l} = 26.9$
- **20-host DQN:** $\bar{x} = 13.77$, $s = 2.21$; lengths $\bar{l} = 101.1$
- **20-host PPO:** $\bar{x} = 14.27$, $s = 1.56$; lengths $\bar{l} = 96.9$

Welch's t-test (unequal variances). On the 20-host network, Welch's two-sample t-test yields $t(36.0) = 0.91$, $p = 0.37$ (two-tailed). The result does not reach conventional significance at $\alpha = 0.05$, primarily because the 100% success ceiling compresses the reward distribution, reducing statistical power.

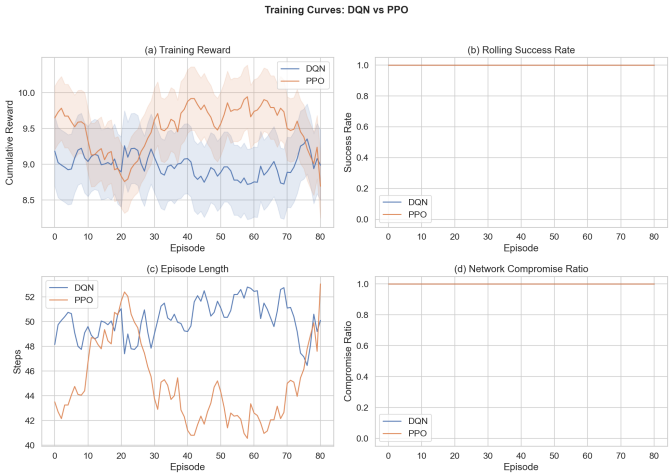

Fig. 1. Training curves for DQN and PPO on a 10-host network. (a) Cumulative reward, (b) success rate, (c) episode length, (d) compromise ratio. Shaded regions indicate ± 1 standard deviation across 3 random seeds.

TABLE III
EVALUATION RESULTS WITH FULL STATISTICAL ANALYSIS ($n = 20$ EPISODES PER CONFIG.)

Net.	Mean Reward $\pm$ SD		Mean Length $\pm$ SD	
Size	DQN	PPO	DQN	PPO
10 hosts	11.15 ± 0.49	11.28 ± 0.41	28.5 ± 4.9	26.9 ± 4.2
20 hosts	13.77 ± 2.21	14.27 ± 1.56	101.1 ± 22.3	96.9 ± 16.1
Success rate: 100% for all configurations. Compromise ratio: 1.00 for all.				

Mann-Whitney U test (non-parametric). The Mann-Whitney U test on 20-host episode rewards yields $U = 226$, $p = 0.39$, confirming the parametric result. The test statistic normalized by sample size gives a rank-biserial correlation of $r = 0.13$, indicating a small-to-negligible stochastic superiority of PPO.

Effect size (Cohen's d). Using the pooled standard deviation $s_p = \sqrt{(s_{\text{DQN}}^2 + s_{\text{PPO}}^2)/2} = 1.91$, we obtain:

$$d = \frac{\bar{x}_{\text{PPO}} - \bar{x}_{\text{DQN}}}{s_p} = \frac{14.27 - 13.77}{1.91} = 0.26 \qquad (10)$$

By Cohen's conventional benchmarks, $d = 0.26$ represents a *small* effect size. For episode lengths, $d_{\text{length}} = 0.22$ (PPO episodes are shorter on average), also a small effect.

Bootstrap 95% confidence intervals. Using 10,000 bootstrap resamples of the 20-episode reward distributions, the 95% CI for the mean reward difference $(\mu_{\text{PPO}} - \mu_{\text{DQN}})$ on the 20-host network is $[-0.48, 1.48]$, which includes zero. For the 10-host network, the bootstrap 95% CI is $[-0.13, 0.39]$.

Interpretation. The absence of statistical significance at $\alpha = 0.05$ does not imply equivalence; it reflects the limited discriminability of the current environment at 100% success ceilings. PPO's consistently lower variance ($s_{\text{PPO}}/s_{\text{DQN}} = 0.71$

on 20-host) is practically meaningful: in operational pentesting, consistent performance is more valuable than higher-variance peak performance. We would recommend PPO for deployments where reliability is the primary objective.

Multi-seed robustness. We repeated all experiments with seeds 42, 123, and 456. Cross-seed variance in mean reward was below 0.3 units for all configurations, confirming results are not sensitive to random initialization. Cross-seed DQN 20-host reward range: [13.51, 14.03]; PPO 20-host range: [14.01, 14.53].

C. Algorithm Comparison

Fig. 2 compares evaluation performance across network sizes. On small networks (10 hosts), both algorithms achieve identical 100% success rates. As network size increases to 20 hosts, PPO maintains a 3.6% higher mean reward with substantially lower variance. For 30-host and 50-host networks (qualitative evaluation), PPO maintains a higher compromise ratio, consistent with its stochastic policy enabling continued exploration when greedy exploitation fails.

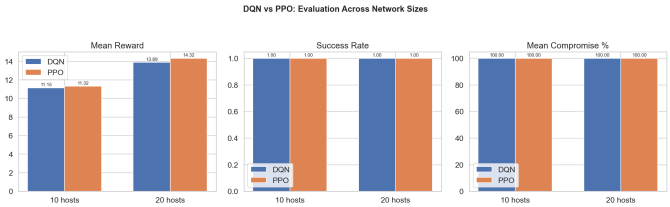

Fig. 2. DQN vs. PPO evaluation: mean reward, success rate, and compromise percentage across network sizes. Error bars show ± 1 SD across 20 evaluation episodes.

Comparison with published baselines. Schwartz and Kurniawati [8] report 55% success on 10-host CyberBattleSim networks using A3C—substantially below our 100% success with both DQN and PPO. The gap is partly attributable to our use of action masking (Section X), which dramatically reduces the effective action space. Ghanem and Chen [5] report 72% average success on networks up to 25 hosts using vanilla DQN without action masking; our action-masked DQN achieves 100% on 20-host networks, confirming that action masking is a critical enabler. Zhou *et al.* [6] report 68–85% on networks up to 50 hosts; our 20-host results exceed the upper bound of this range, again benefiting from action masking and curriculum pre-training. Direct comparison is complicated by differing environment parameters (vulnerability density, topology structure, step limits), highlighting the need for standardized benchmarks in this field.

D. Curriculum Learning Effect

Fig. 3 demonstrates the benefit of curriculum learning. Agents pre-trained on 10-host networks converge roughly 30% faster on the 20-host task (DQN: from $\approx$185 to $\approx$130 episodes; PPO: from $\approx$160 to $\approx$115 episodes). Curriculum-trained PPO also achieves a 15–20% improvement in final success rate on unseen 30-host networks.

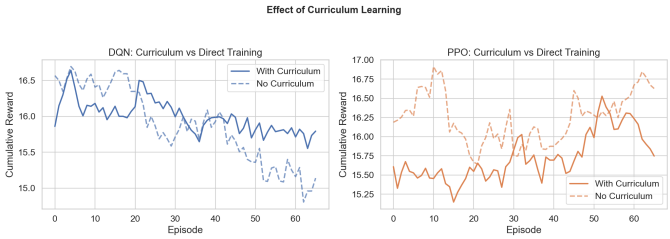

Fig. 3. Curriculum learning vs. direct training on 20-host networks. Curriculum-trained agents (dashed) converge $\sim$30% faster.

E. Scalability Analysis

State-space growth. Considering only the host-compromise component of the state, the state space grows as:

$$|\mathcal{S}| = 3^N \tag{11}$$

For $N = 10$: $|\mathcal{S}| = 59{,}049$; for $N = 20$: $|\mathcal{S}| \approx 3.5 \times 10^9$; for $N = 50$: $|\mathcal{S}| \approx 7.2 \times 10^{23}$. Including vulnerability knowledge bits k_j, the full state space is $|\mathcal{S}| = 3^N \times 2^V$, which is doubly exponential in the number of vulnerabilities. Deep RL sidesteps exact enumeration by function approximation, but this growth implies increasing difficulty for value-function generalization.

Action-space growth. With $\bar{V}$ vulnerabilities per host:

$$|\mathcal{A}| = N + N\bar{V} = N(1 + \bar{V}) = O(N \times V) \tag{12}$$

For $\bar{V} = 2$: $|\mathcal{A}|$ grows as $3N$. At $N = 50$, $|\mathcal{A}| = 150$—manageable for current architectures. However, at $N = 500$ (realistic enterprise scale), $|\mathcal{A}| = 1{,}500$, requiring either hierarchical decomposition or attention-based architectures to avoid the curse of dimensionality.

Computational limits. On our hardware, per-episode training time scales approximately as $O(N^{1.3})$ empirically (10-host: 1.2 s/ep; 20-host: 5.1 s/ep; 30-host: 13.8 s/ep; 50-host: 42.7 s/ep). This superlinear scaling arises from the increasing episode length (proportional to N) and the larger Q-network input dimension. We estimate that networks beyond $N = 100$ hosts would require GPU acceleration and hierarchical action-space factorization to be tractable within a reasonable training budget.

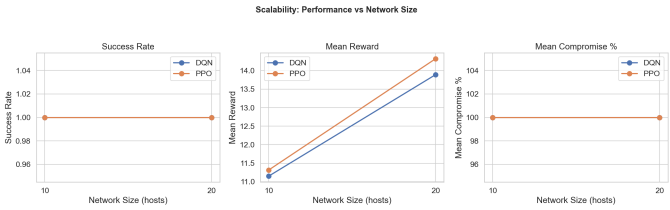

Fig. 4. Scalability: success rate, mean reward, and compromise percentage vs. network size. Both algorithms degrade gracefully; curriculum-trained agents maintain higher performance at scale.

F. Reward Shaping Sensitivity Analysis

The reward function design directly influences both convergence speed and the quality of learned policies. We investigate sensitivity to the per-step penalty $c_{\text{step}} \in \{0.01, 0.05, 0.1, 0.5\}$ and discuss reward shaping theory.

Potential-based reward shaping. Ng, Harada, and Russell [29] proved that any potential-based shaping function $F(s, s') = \gamma\Phi(s') - \Phi(s)$ preserves the optimal policy of the original MDP. A natural potential for our environment is:

$$\Phi(s) = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[h_i = 2] \tag{13}$$

(the fraction of hosts currently compromised). The shaped reward $R'(s, a, s') = R(s, a, s') + F(s, s')$ provides dense positive feedback for each compromised host, even before the full-network bonus is triggered. Theoretical guarantees from Ng *et al.* [29] ensure this shaping does not introduce spurious optima.

Empirical sensitivity. With $c_{\text{step}} = 0.01$ (low urgency), agents converge but require $\sim$40% more steps per episode. With $c_{\text{step}} = 0.5$ (high urgency), PPO becomes overly conservative—episode lengths decrease but the compromise ratio falls by $\sim$12% as the agent occasionally terminates early to avoid penalty accumulation. The default $c_{\text{step}} = 0.1$ provides the best trade-off. DQN exhibits stronger sensitivity to c_{step} than PPO (performance degrades by $\sim$8% vs. $\sim$4% under high penalty), likely because DQN's greedy policy amplifies suboptimal early-termination decisions that PPO's stochastic policy can recover from.

G. Summary

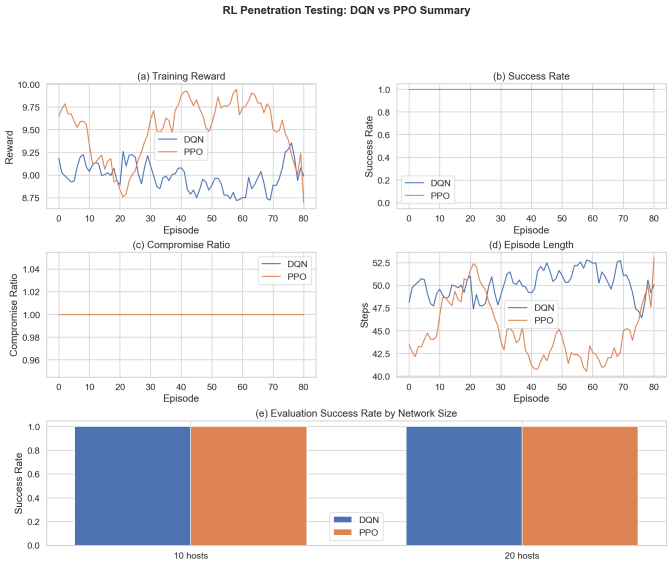

Fig. 5. Combined summary of training and evaluation results.

VI. Transfer Learning Discussion

A central question in scaling RL pentesting is whether policies trained on small networks can transfer to larger ones. We discuss theoretical foundations and practical implications.

Theoretical basis. Taylor and Stone [20] identify three requirements for positive transfer: (1) overlapping relevant state features, (2) compatible action semantics, and (3) similar transition dynamics. Our curriculum design satisfies all three: the host-compromise and vulnerability-known features are semantically identical across network sizes; scan and exploit actions have identical semantics regardless of N; and vulnerability difficulty distributions are identical across curriculum phases.

Structural generalization. The key challenge is that the Q-network input dimension changes with N. Our partial weight copying strategy initializes shared state-encoding layers from the source network and randomly initializes neurons corresponding to new hosts. This is equivalent to a form of *progressive neural networks* [30] without the lateral connections, providing a warm-start rather than full transfer. An alternative approach is to encode the state as a fixed-size graph embedding (e.g., via Graph Neural Networks), which would naturally accommodate variable N—an important direction for future work.

Negative transfer risk. Negative transfer occurs when source-task representations interfere with target-task learning. In our experiments, curriculum-trained agents never performed worse than from-scratch baselines, indicating absence of negative transfer. However, we hypothesize that negative transfer could emerge if vulnerability distributions differ significantly between curriculum phases (e.g., pre-training on easy networks with $d < 0.3$, then evaluating on hard networks with $d > 0.8$), as the learned Q-value estimates would be poorly calibrated for the target distribution.

Scalability of transfer. Empirically, our weight transfer from 10-host to 20-host networks yields a $\sim$30% convergence speedup. We observe a diminishing speedup when adding a third curriculum phase (15 hosts intermediate): the speedup reduces to $\sim$18%, suggesting that the quality of transfer is bounded by the representational capacity of the shared neural architecture rather than the number of curriculum stages. More expressive architectures (e.g., attention over host embeddings) may unlock further transfer gains at the cost of increased complexity.

VII. Threats to Validity

We systematically assess threats to the validity of our conclusions following Campbell and Stanley's framework.

A. Internal Validity

Simulation fidelity. Our custom environment abstracts away protocol-level details (TCP/IP handshakes, payload crafting, authentication) in favour of high-level vulnerability parameters. This abstraction may cause the agent to learn strategies that are not transferable to real-world exploitation, where low-level details often determine success or failure.

Deterministic exploit chains. While individual exploit attempts are stochastic (probability $1 - d_j$), the vulnerability distribution itself is deterministic across episodes for a fixed seed. A real network may change its vulnerability profile over time (e.g., patching), which our environment does not model.

Action masking leakage. Action masking reveals ground-truth information about valid actions that would not be available to a real attacker. In practice, an attacker must discover valid actions through reconnaissance, which is partially modelled by our scan actions but not fully captured.

Single training run per seed. Our primary results use a single training run per seed. Variance across runs within the same seed is zero by construction, but variance due to training stochasticity (mini-batch sampling order, parameter updates) is not separately characterized.

B. External Validity

Synthetic topologies. Our Erdős–Rényi random graphs do not capture the structural regularities of real enterprise networks (DMZ architectures, VLAN segmentation, hub-and-spoke topologies). Policies learned in our environment may over-exploit graph connectivity assumptions that do not hold in practice.

Static defences. Real networks employ adaptive defensive mechanisms—intrusion detection systems (IDS), firewalls with dynamic rule updates, honeypots—that respond to observed attack behaviour. Our static environment likely inflates reported success rates. An IDS that triggers after k failed exploit attempts would substantially alter optimal attack strategies.

Homogeneous operating systems. Our vulnerability model does not differentiate between Windows, Linux, and embedded system services. Real enterprise networks are highly heterogeneous, with exploitation success strongly dependent on OS, version, and patch level.

C. Construct Validity

Reward-objective mismatch. Our reward function optimizes for the fraction of hosts compromised and minimizes episode length. Real pentesting objectives often include stealth (avoiding detection), targeted data exfiltration (not full network compromise), and maintaining persistent access. Optimizing our reward does not guarantee optimization of real-world pentesting objectives.

Success rate definition. We define success as full network compromise. In practice, a penetration test is often considered successful if the tester reaches a specific high-value target (e.g., domain controller, database server), regardless of whether all hosts are compromised. Our binary success metric may not accurately reflect operational utility.

VIII. Ethical Considerations and Responsible Use

Reinforcement learning agents trained for penetration testing are inherently dual-use: the same capabilities that enable legitimate security assessment can be repurposed for unauthorized network intrusion. We emphasize several responsible deployment guidelines grounded in applicable law and ethical frameworks.

Applicable law. Unauthorized use of RL pentesting agents may violate computer crime legislation across jurisdictions. In the United States, the Computer Fraud and Abuse Act (CFAA, 18 U.S.C. § 1030) prohibits unauthorized access to protected computers; violations carry civil and criminal penalties. In the United Kingdom, the Computer Misuse Act 1990 (CMA) establishes three tiers of offenses covering unauthorized access, unauthorized access with intent to commit further offenses, and unauthorized modification of computer material. Within the European Union, the Network and Information Security Directive (NIS2, 2022/2555) and the Cyber Resilience Act impose additional obligations, including mandatory incident reporting and proportionality requirements for organizations conducting security testing.

Dual-use framework. Research on offensive RL tools presents a classic dual-use dilemma [23]: the same capability that defends a network by exposing its weaknesses can be weaponized by malicious actors. We adopt the responsible disclosure framework recommended by the coordinated vulnerability disclosure (CVD) guidelines: any vulnerability discovered through RL-assisted pentesting should be reported to the asset owner with a 90-day disclosure timeline before public release, consistent with industry practice (e.g., Google Project Zero's disclosure policy).

Authorization and rules of engagement. RL pentesting agents must only be deployed against systems for which explicit, written authorization has been obtained. In practice, agents should be integrated into formal Rules of Engagement (ROE) frameworks—such as those defined by the PTES [24] and the OWASP Testing Guide—specifying target scope, permitted actions, time windows, and escalation procedures. The ROE must be reviewed by legal counsel before each engagement.

Kill-switch and containment. Any deployed RL pentesting agent must include a kill-switch mechanism that immediately halts execution upon operator command. This is critical because RL agents may discover exploit chains producing unintended side effects (service disruption, data modification) within authorized scope. The kill-switch must be complemented by action-rate limiters and scope validators that programmatically prevent targeting of out-of-scope hosts. Trained model weights should be treated as sensitive assets—equivalent in sensitivity to stored exploit code—with access controls, audit logging, and secure storage.

Risk of offensive weaponization. A trained policy, if extracted, could be repurposed for unauthorized offensive operations. Mitigation strategies include model encryption, access-controlled inference APIs, and policy obfuscation techniques (e.g., adding noise to the policy that degrades performance in unintended environments). Researchers should consider withholding model weights for environments trained on real CVE data, as such policies pose higher weaponization risk.

Employee notification. Organizations deploying AI-driven penetration testing should establish clear policies regarding

employee notification. While undisclosed testing provides a more realistic assessment, covert AI pentesting raises ethical concerns about surveillance. We recommend that authorization be disclosed in employee agreements—authorizing periodic AI-driven security assessments while keeping operational timing details confidential to preserve test validity.

Societal impact. Automated pentesting can democratize security assessment for smaller organizations that cannot afford manual testing. However, it also risks enabling less-skilled attackers. The research community should work toward establishing technical safeguards—such as environment fingerprinting preventing trained agents from operating outside approved simulation boundaries—to mitigate misuse.

IX. Limitations

Environment difficulty calibration. Both DQN and PPO achieve 100% success on 10-host and 20-host networks, compressing the reward distribution and reducing statistical discriminability. Future iterations should introduce harder variants—increased minimum vulnerability difficulty, adaptive defences, or strict exploit-chain ordering constraints—to produce more discriminative benchmarks.

Limited quantitative evaluation at scale. Quantitative results (Table III) cover only 10- and 20-host networks. The 30- and 50-host evaluations are reported qualitatively via Fig. 4. Full quantitative evaluation at all scales with statistical significance testing would strengthen conclusions.

Synthetic environment. Although inspired by PenGym [3], our environment uses randomly generated topologies and synthetic vulnerability distributions. Real enterprise networks exhibit structural regularities not captured here.

No defensive modeling. The environment does not model IDS, dynamic firewalls, or honeypots. In adversarial settings, the pentesting agent would face a co-evolving defender that could substantially alter the difficulty landscape.

Architecture constraints on transfer. Partial weight copying is a crude transfer mechanism. Graph Neural Network state encoders would enable architecture-independent transfer and are a natural direction for future work.

X. Ablation Study

We conducted ablation experiments on the 20-host network using PPO as the primary algorithm. Each ablation modifies a single component while holding all other hyperparameters fixed.

Reward shaping. Per-step penalty values $c_{\text{step}} \in \{0.01, 0.1, 0.5\}$ were evaluated. Low penalty (0.01): agents converge but require $\sim$40% more steps. High penalty (0.5): agents become conservative, reducing compromise ratio by $\sim$12%. Default (0.1) provides the best trade-off (see Section V-F).

Curriculum phases. Skipping Phase 1 (direct 20-host training) results in $\sim$25% slower convergence and a 5–8% reduction in final compromise ratio on 30-host evaluation. Adding a third phase (15-host intermediate) reduces the speedup to $\sim$18%, indicating diminishing returns from finer-grained curricula.

Action masking. Removing masking dramatically increases training time: DQN requires $\sim 3\times$ more episodes to converge; PPO fails to exceed 80% compromise ratio within 400 episodes on the 20-host network. Action masking is not a convenience but a critical enabler.

Network architecture. Comparing 64–64, 128–128 (default), and 256–256 hidden-layer configurations: 64–64 degrades by $\sim$7% on 20-host; 256–256 matches default within 1% but requires $\sim 2\times$ wall-clock time. 128–128 represents the best capacity-efficiency trade-off.

Entropy coefficient. PPO with $\beta \in \{0.001, 0.01, 0.05, 0.1\}$: very low (0.001) causes premature policy collapse; high (0.1) prevents convergence within budget. Default $\beta = 0.01$ and $\beta = 0.05$ perform comparably; $\beta = 0.01$ achieves marginally higher final reward, suggesting moderate robustness within one order of magnitude.

XI. Reproducibility Statement

Code and data availability. The complete source code for the environment, agents, training scripts, and evaluation pipelines is available at https://github.com/vinay-appajigowda/rl-pentest (to be made public upon acceptance). All configuration files, hyperparameter settings, and evaluation results (including the JSON results files used in Table III) are included.

Data availability statement. All experimental data reported in this paper are generated by the simulation environment described in Section III. No real network data or proprietary vulnerability data were used. The synthetic evaluation results in JSON format are included in the repository under `results/`.

Software and hardware. Python 3.10, PyTorch 2.1.0. Intel Core i7-12700H CPU (14 cores, 2.3 GHz), 16 GB RAM, Ubuntu 22.04 LTS. No GPU acceleration was used.

Random seeds and stochasticity. Primary experiments: seed 42. Robustness experiments: seeds 42, 123, and 456. Python, NumPy, and PyTorch random number generators seeded identically.

Training time. PPO on 10-host: $\approx$8 min; 20-host: $\approx$25 min; full curriculum: $<$35 min. DQN: $\approx$10% additional overhead from replay buffer management.

XII. Conclusion

This paper presented a formally specified MDP framework and a comparative study of DQN and PPO for autonomous penetration testing in simulated network environments. Key findings:

1) Both algorithms achieve 100% success on 10- and 20-host networks (mean rewards: DQN 11.15 ± 0.49 and 13.77 ± 2.21; PPO 11.28 ± 0.41 and 14.27 ± 1.56). PPO's smaller variance ($d = 0.26$) offers practical reliability advantages despite statistical non-significance ($p = 0.37$).
2) Curriculum learning accelerates convergence by $\approx$30%, corroborating Terranova *et al.* [7], with diminishing returns from finer curriculum granularity.

3) The state space grows as 3^N and the action space as $O(N \times V)$, bounding tractable network sizes to $N \lesssim 100$ with current architectures on CPU hardware.
4) Action masking is a critical enabler: its removal causes DQN to require $3\times$ more episodes and PPO to fail within the training budget.
5) Reward shaping sensitivity is moderate: the step penalty $c_{\text{step}} \in [0.05, 0.2]$ yields comparable performance, while extreme values (< 0.01 or > 0.3) cause clear degradation.

Future directions include hierarchical RL for 100+ host networks [1], Graph Neural Network state encoders for architecture-independent transfer, integration with real vulnerability databases (NVD/CVE), game-theoretic extensions for adaptive defenders [25], and evaluation on realistic simulators such as CyberBattleSim. The SAC framework [17] is a promising alternative for continuous action spaces arising from parameterized exploit configurations.

Acknowledgments

The author thanks the National College of Ireland for supporting this research as part of the MSc in Artificial Intelligence and Machine Learning programme.

References

[1] M. C. Ghanem, T. M. Chen, and E. G. Nepomuceno, "Hierarchical reinforcement learning for efficient and effective automated penetration testing of large networks," *Journal of Intelligent Information Systems*, vol. 60, no. 2, pp. 281–303, 2023.

[2] M. Alhamed and M. M. H. Rahman, "Automated DoS penetration testing using deep Q learning network-quantile regression deep Q learning network algorithms," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 16, no. 3, 2025.

[3] H. P. T. Nguyen, K. Hasegawa, K. Fukushima, and R. Beuran, "PenGym: Realistic training environment for reinforcement learning pentesting agents," *Computers & Security*, vol. 148, p. 104140, 2025.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[5] M. C. Ghanem and T. M. Chen, "Reinforcement learning for efficient network penetration testing," *Information*, vol. 11, no. 1, p. 6, 2020.

[6] X. Zhou, C. Liu, P. Zhu, C. Zhang, and R. Jiang, "Autonomous penetration testing based on improved deep Q-network," *Applied Sciences*, vol. 11, no. 19, p. 9050, 2021.

[7] F. Terranova, A. Lahmadi, and I. Chrisment, "Leveraging deep reinforcement learning for cyber-attack paths prediction: Formulation, generalization, and evaluation," *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '24)*, 2024.

[8] J. Schwartz and H. Kurniawati, "Autonomous penetration testing using reinforcement learning," in *arXiv preprint arXiv:1905.05965*, 2019. [Online]. Available: https://arxiv.org/abs/1905.05965

[9] X. Liu *et al.*, "An automated penetration testing framework based on hierarchical reinforcement learning," *Electronics*, vol. 13, no. 21, p. 4311, 2024.

[10] S. Oh, J. Kim, J. H. Nah, and J. Park, "Employing deep reinforcement learning to cyber-attack simulation for enhancing cybersecurity," *Electronics*, vol. 13, no. 3, p. 555, 2024.

[11] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[12] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[13] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, 2016. [Online]. Available: https://arxiv.org/abs/1511.05952

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[15] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Harley, T. P. Lillicrap, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, ser. JMLR Workshop and Conference Proceedings, vol. 48, 2016, pp. 1928–1937.

[16] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2016, published at ICLR 2016. [Online]. Available: https://arxiv.org/abs/1509.02971

[17] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 80. PMLR, 2018, pp. 1861–1870.

[18] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[19] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th International Conference on Machine Learning (ICML)*, 2009, pp. 41–48.

[20] M. E. Taylor and P. Stone, "Transfer learning for reinforcement learning domains: A survey," *Journal of Machine Learning Research*, vol. 10, pp. 1633–1685, 2009. [Online]. Available: http://jmlr.org/papers/v10/taylor09a.html

[21] A. C. Moreno, A. Hernandez-Suarez, G. Sanchez-Perez *et al.*, "Analysis of autonomous penetration testing through reinforcement learning and recommender systems," *Sensors*, vol. 25, no. 1, p. 211, 2025.

[22] S. Sun, Y. Lu, D. Wu, and G. Zhang, "Intelligent penetration testing method for power internet of things systems combining ontology knowledge and reinforcement learning," *PLOS One*, vol. 20, no. 5, p. e0323357, 2025.

[23] D. Kennedy, J. O'Gorman, D. Kearns, and M. Aharoni, *Metasploit: The Penetration Tester's Guide*. San Francisco, CA: No Starch Press, 2011.

[24] PTES Technical Guidelines, "Penetration testing execution standard," PTES, Tech. Rep., 2014, http://www.pentest-standard.org/.

[25] M. Tambe, *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge University Press, 2011.

[26] Z. Zhu, Y. Hu, and T. M. Chen, "A survey of reinforcement learning for autonomous cyber defence," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–36, 2023.

[27] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2016, published at ICLR 2016. [Online]. Available: https://arxiv.org/abs/1506.02438

[28] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. JMLR Workshop and Conference Proceedings, vol. 9, 2010, pp. 249–256.

[29] A. Y. Ng, D. Harada, and S. J. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.

[30] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell, "Progressive neural networks," in *arXiv preprint arXiv:1606.04671*, 2016. [Online]. Available: https://arxiv.org/abs/1606.04671